

Python Desktop Poker Development

CustomTkinterを用いたデスクトップアプリ開発

中間制作報告：プログラミング課題

制作物の概要



カリキュラムで習得した知識の集大成としての実装

コンセプトと要件定義

開発コンセプト

「Pythonの基礎知識を活用し、視覚的に楽しめる実用的なアプリケーションの構築」

オブジェクト指向(OOP)を基盤に、拡張性の高いコード設計を目指しました。

主要な要件

- 直感的なGUI
- 高度な役判定アルゴリズムの実装
- Excelによる戦績データの保存・管理
- ダブルアップ等のゲーム性向上機能

システム構成とクラス設計

クラスベースの設計

責務を明確に分けた4つの主要クラス：

Card / Deck: カード情報と山札の管理

Player: 手札とチップ、役判定のロジック

PokerApp: GUI制御と全体のゲーム進行

各クラスが独立しているため、ロジックの修正がGUIに影響を与えない疎結合な設計です。

```
class Card:
    def __init__(self, suit, value):
        self.suit = suit
        self.value = value

class Deck:
    def __init__(self):
        self.cards = [Card(s, v) ...]
    def shuffle(self): ...
    def draw(self): ...

class Player:
    def __init__(self):
        self.hand = []
        self.chips = 1000
    def judge_hand(self): ...

class PokerApp(ctk.CTk):
    def __init__(self):
```


オブジェクト指向設計の実践



カプセル化

Cardクラスは属性を保持し、Deckクラスは「シャッフル」等の振る舞いをメソッド内に隠蔽しています。

```
def draw(self): return self.cards.pop()
```



責務の分離

Playerがデータ判定、PokerAppがGUI制御を担当。UIとロジックを分離し、可読性を高めています。



状態の管理

インスタンス変数を活用し、チップ残高や手札の状態をラウンドを跨いで正確に保持しています。

役判定アルゴリズムの解説

```
def judge_hand(self):
    """手札の役とキッカー情報を判定"""
    numbers = sorted(
        [c.value for c in self.hand],
        reverse=True
    )
    counts = Counter(numbers)

    sorted_counts = sorted(
        counts.items(),
        key=lambda x: (x[1], x[0]),
        reverse=True
    )
    compare_list = []
    for num, count in sorted_counts:
        compare_list.extend([num] * count)

    count_pattern = sorted(counts.values())
```

judge_hand() メソッドのコアロジック

- 🔗 **リスト内包表記とCounter:** 手札の数値を集計し、count_patternとして重複度をパターン化。
- ↓🔍 **役の優先度判定:** is_flushやストレート判定の真偽値を用いて、最上位の役から順に条件分岐 (if-elif) で特定。
- 🔗 **A-5ストレート対応:** 最小のストレート (5-4-3-2-A) というポーカー固有の例外規則も論理式でカバー。

GUI : CustomTkinter

モダンなデスクトップUI

従来のTkinterにはない、洗練された外観を実装：

 **Dark Theme** によるモダンな配色

 視覚的な動的色彩設計

 階層的なカード表示レイアウト

 グリッドによる正確な位置制御

ユーザー体験と視覚フィードバック



カード交換

チェックボックスによる直感的な操作とUI上の即時更新を連動。



結果通知

勝敗に応じた配色でユーザーへ状況を伝達。



ベット管理

不正入力を防ぐバリデーション付き入力制御。

ダブルアップ機能の設計

x2

High or Low Logic

別ウィンドウによるミニゲーム

ゲーム性を高める追加機能：

基準カードに対する大小予想ロジック。

的中による配当2倍、または没収の条件分岐。

grab_set() によるモーダルウィンドウ制御。

データ永続化：Excel連携

openpyxlによるログ管理

データの蓄積による戦績分析：

項目	保存されるデータ
日時	タイムスタンプ
ベット/役	各ラウンドの技術的詳細
勝敗	ディーラーとの比較結果
所持チップ	残高の推移記録

📄 poker_history.xlsx				
	A	B	C	D
1	Timestamp	Bet	HandRole	TotalChips
2	2026-06-19 15:30:12	100	ツーペア	1200
3	2026-06-19 15:32:45	200	フルハウス	2800
4	2026-06-19 15:35:01	150	役なし	2650

役の配当オッズ設定 (HAND_ODDS)



配当バランスの設計

ゲームバランスを考慮した倍率定義：

出現率に基づいた指数的な配当設定。

ベット額に連動した配当計算。

開発における課題と試行錯誤



設計方針の模索

目標が曖昧で、何から作るべきかの優先順位に迷いが生じました。機能の着地点を定めないまま開始したため、発展性に課題を感じる場面もありました。



思考のプロセス管理

多角的に設計を検討するため、**archiveフォルダ**を活用。過程をログとして保存し、論理構造に至る変遷を管理しました。



段階的進化

関数ベースからオブジェクト指向への移行を通じ、コードを洗練させることの重要性を学びました。

今後の改善案とさらなる発展



テキサスホールデムへの換装

現在の「5枚ドロークポーカー」から、コミュニティカードを使用する「テキサスホールデム」形式へゲームシステムを変更し、戦略性を高めます。



オンライン化への展望

インターネットを使って、離れた場所にいる他のプレイヤーとリアルタイムで対戦ができる仕組みを作りたいと考えています。



CPUアルゴリズムの強化

ディーラー側の戦略に「ブラフ（はったり）」などの要素を加え、より高度で予測しにくい対戦相手の実装を目指します。

カリキュラム学習の成果

変数・制御構文から外部ライブラリ活用まで
「実力として定着した知識」の証明

Thank You

ご清聴ありがとうございました。

使用素材・技術一覧

著作権および素材に関する確認

本スライドで使用されているすべての素材は、以下の通りフリーおよびオープンソース素材のみで構成されています：

- ✓ **フォント:** Google Fonts (Noto Sans JP, Roboto) - SIL OFL
- ✓ **アイコン:** FontAwesome Free - SIL OFL / MIT License
- ✓ **画像:** CSS描画およびコードビューを使用しており、第三者の著作権画像は含まれていません。
- ✓ **ライブラリ:** openpyxl, CustomTkinter (MIT License)